
How Much Should the Validator Say? Feedback Granularity for Symbolic Plan Repair with a Small Open LLM

Qinzheng Ma
Rice University
qm18@rice.edu

Shichen Tang
Rice University

Jialin Wu
UC San Diego

Abstract

Generate-validate-repair loops for planning rely on a sound symbolic verifier, and studies with GPT-4 concluded that the content of the verifier’s feedback matters little because the verifier mainly acts as a stopping criterion. We test whether this holds for a frozen 4B open model at inference time. Qwen3-4B-Instruct-2507 writes one-shot PDDL plans for 132 instances of Blocksworld, an obfuscated Mystery Blocksworld, Gripper and Logistics; a STRIPS executor validates them, and every failure enters up to three repair rounds under five arms: self-verification without validator information, a binary verdict, the first failing action with its unmet preconditions, that plus all facts of the failing state, and that plus the actions applicable in it. One-shot success is 4.5%, and no arm repairs more than 2.4% of the failures; the primary contrast, localized versus binary feedback, is +0.8 points (95% CI [0.0, 2.4]). Below the success threshold the arms do differ: localized feedback cuts plan repetition from 44% to 6% of turns and doubles the rate at which the next attempt gets past the reported failing action (14.5% versus 7.2%); listing the applicable actions raises that rate to 54%, but the revised plans fail one or two actions later. Full-state feedback costs about 45 extra prompt tokens per round, while self-verification triples output length and outputs facts parsed as actions. For a small model the validator’s message is used locally but cannot supply the planning competence that is missing.

1 Introduction

LLM-Modulo planning pairs a plan-proposing language model with a sound critic that rejects invalid plans and reports back [Kambhampati et al., 2024, Valmeekam et al., 2023c]; for an embodied agent the natural critic is a symbolic world model in PDDL [McDermott et al., 1998]. When a plan fails, the model can be told a single bit (invalid), the first inapplicable action and its missing preconditions, as in robot re-prompting [Raman et al., 2024], or every fact then true, as an agent would observe after a failed step. How much to say is a design knob that also sets context cost per round.

With GPT-4 and VAL, Valmeekam et al. [2023a] compared binary, first-error and all-errors feedback on Blocksworld, and Stechly et al. [2024] added Mystery Blocksworld. Both read the differences as small (the second: the verifier mainly stops repeated guessing) and found self-critique far below a sound verifier. For small open models the only evidence, from training, points the other way: detailed validator feedback beat binary when instruction-tuning Llama-3-8B on three of our domains [Verma et al., 2025]. Frozen open models rarely produce valid plans, almost never on Mystery Blocksworld [Vyas et al., 2025, Valmeekam et al., 2024], so guessing has little to work with; whether a richer message helps at inference time is, to our knowledge, untested.

We ask this for a frozen 4B open model, Qwen3-4B-Instruct-2507 [Yang et al., 2025], on one 24GB consumer GPU. Our STRIPS executor (pyperplan grounding, Alkhazraji et al., 2020) validates a one-shot plan (round 0); each failure gets up to three repair rounds under five arms: SELF (self-verification), BIN (invalid only), ACT (first failing action, unmet preconditions or goals), STATE (plus the facts true at failure) and APPL (plus the applicable actions). Round 0 is shared, so comparisons are paired. The 132 instances are IPC and PlanBench Blocksworld, the same with names obfuscated (Mystery), Gripper and Logistics [Valmeekam et al., 2023b, McDermott, 2000]. The primary contrast is ACT versus BIN pooled over domains; everything else is exploratory (metrics in Section 2).

Contributions:

- A paired replication of the feedback-granularity ablation at 4B scale: one-shot success is 4.5%, three repair rounds add at most three instances under any arm, and localized minus binary feedback is +0.8 points (95% CI [0.0, 2.4]). The GPT-4 conclusion that content matters little holds here for a different reason: there is almost nothing to stop on.
- Two levels absent from the prior comparisons, the full failing state and the set of applicable actions, weighed against their context cost: they repair 1.6% and 2.4% of failures for about 45 and 75 extra prompt tokens per round, and listing the applicable actions raises the rate at which the next attempt clears the reported action from 16% to 54% without raising success.
- Evidence that the message is nevertheless used: localized feedback cuts verbatim plan repetition from 44% to 6% of turns and doubles the rate at which the next attempt clears the reported action, while self-verification triples output length and derails into prose.

Related work. Other pipelines have the LLM formalize problems for a classical planner [Liu et al., 2023], write domain models corrected by a validator [Guan et al., 2023], or synthesize planning programs debugged with execution errors [Silver et al., 2024]. Validators naming the first faulty action were compared with LLM self-validation [Zhou et al., 2024], and recent Blocksworld symbolic-feedback loops use proprietary models or a 32B open model with one feedback format [Zhang et al., 2026, Nukapotula et al., 2026]. SELF follows Self-Refine [Madaan et al., 2023]; the ladder mirrors self-debugging in code [Chen et al., 2024].

2 Experimental Setup

2.1 Tasks and data

Four STRIPS families [McDermott et al., 1998]. *Blocksworld*: 17 IPC instances with 4 to 9 blocks [McDermott, 2000] and 39 four-block PlanBench instances [Valmeekam et al., 2023b]. *Mystery Blocksworld*: the same 56 instances with every action and predicate renamed to a nonsense word (our own mapping, not PlanBench’s). *Gripper* (4) and *Logistics* (16), also IPC. We keep instances whose reference plan has at most 45 actions. Reference plans: pyperplan [Alkhazraji et al., 2020] A* with LM-cut (40 s cap), else greedy best-first search with h^{FF} (60 s cap); 124 of 132 are optimal. Each family’s shortest-plan instance is the one-shot demonstration and is not evaluated (Appendix B).

2.2 Planning loop and feedback levels

Round 0 is one-shot generation from a system prompt, task instruction, domain, one demonstration with its plan, and the problem (comment-stripped PDDL), one parenthesized action per line. It is generated once, reused verbatim by every arm (NONE), and its failures form the repair-eligible set E . Each arm gives every instance in E up to three repair rounds: a user turn carries the arm’s message (Table 1); the loop stops at the first valid plan and keeps the full history. Validator messages (ACT, STATE, APPL) use the vocabulary the model sees (obfuscated in Mystery) and never contain a reference plan; STATE adds facts only for precondition and goal failures and otherwise equals ACT. APPL, added after the first four arms had run, is the closest the validator can come to a suggestion without planning (Appendix A).

2.3 Validator and metrics

Success is judged by our STRIPS executor on the pyperplan-grounded task. Errors are labelled in first-failure-wins order: unparseable, unknown action, wrong arity, unknown object (merged

Arm	Content of the repair message	Validator
NONE	no repair; the round-0 plan shared by all arms	n/a
SELF	fixed instruction to verify preconditions and effects action by action, check the goal, and fix any error found	no
BIN	“Your plan is invalid.”	yes
ACT	index and name of the first inapplicable action with its unmet preconditions, or the unmet goal literals; specific messages for unparseable output, unknown action, wrong arity, and unknown object or grounding	yes
STATE	ACT plus all true facts in the state where execution failed (just before the failing action, or at the end of the plan)	yes
APPL	STATE plus the grounded actions applicable in that state (at most 40, alphabetical)	yes

Table 1: Feedback arms. Every message ends with the same request for a complete corrected plan from the initial state; last column: whether the message uses validator information.

Table 2: Success (%) of Qwen3-4B-Instruct-2507 by domain. n : instances; $r=0$: one-shot success shared by all arms; $|E|$: instances entering repair; the last five columns give cumulative success after three repair rounds under each feedback level.

Domain	n	$r=0$	$ E $	SELF	BIN	ACT	STATE	APPL
Blocksworld	56	2	55	2	4	4	5	5
Mystery BW	56	2	55	2	2	4	2	4
Logistics	16	0	16	0	0	0	0	0
Gripper	4	100	0	100	100	100	100	100
All	132	5	126	5	5	6	6	7

in reporting with tuples lacking a grounded operator), precondition, goal unmet. Self-test: every reference plan validates and every single-action deletion or single-argument substitution of one is rejected (Appendix C). Metrics: cumulative success by round, conditional repair rate on E , error taxonomy of each arm’s last attempt, length ratio of valid plans to the reference, prompt tokens per round and generated tokens.

2.4 Models and decoding

Qwen3-4B-Instruct-2507 [Yang et al., 2025], bf16, Hugging Face transformers, one 24 GB consumer GPU; chat template with thinking disabled, greedy decoding, at most 420 new tokens (truncated outputs are validated as is), batches of 12 length-sorted conversations, left padding.

2.5 Statistics

Instances are the unit of analysis and contrasts on E are paired. Primary contrast: ACT versus BIN conditional repair rate after round 3, pooled over domains, reported as the paired difference with a 95% instance-level bootstrap percentile interval (10,000 resamples), discordant counts, and an exact binomial test on them. Per-arm rates carry bootstrap intervals. All other contrasts (STATE versus ACT, APPL versus STATE, SELF versus the validator arms) and per-domain or per-length splits are exploratory; Gripper (4 instances) is shown but not interpreted.

3 Results

One-shot planning. The model solves 6 of 132 instances at round 0 (4.5%): 1 of 56 in Blocksworld, 1 of 56 in Mystery, 0 of 16 in Logistics and all 4 Gripper instances. Of the 126 failures, 114 are precondition violations and 12 are tuples with no grounded operator; 76 of the 126 failures occur at the first action, and the mean index of the first failing action is 1.9. The typical error is a pick-up of a block that is not on the table or not clear, so the plans fail before any goal is addressed.

Repair. Three rounds of repair barely move success (Table 2, Figure 1): the conditional repair rate on E is 0.0% for SELF, 0.8% for BIN, 1.6% for ACT, 1.6% for STATE and 2.4% for APPL (bootstrap

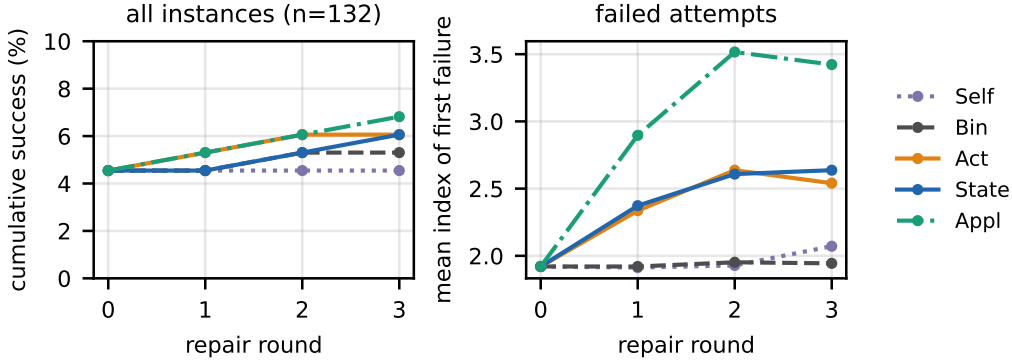


Figure 1: Left: cumulative success by repair round per feedback level, all 132 instances. Right: mean index of the first failing action among invalid attempts; richer feedback moves the failure later without reaching the goal.

95% upper bounds 0.0, 2.4, 4.0, 4.0 and 5.6). The primary contrast ACT–BIN is +0.8 points (95% CI [0.0, 2.4], one discordant instance, exact $p=1$), and STATE–ACT is 0 (one instance each way). Every repaired instance is a 4-block problem; nothing with a reference plan longer than 6 actions is ever repaired. At the level of task success the granularity question therefore has a null answer, and the GPT-4 picture of a validator as a stopping criterion for repeated attempts does not carry over: there is rarely a valid plan to stop on.

What the feedback changes. The arms differ clearly below the success threshold. Under BIN the model returns a plan identical to its previous one in 44% of repair turns, and under SELF in 33%; under ACT and STATE it does so in only 6%. Localized feedback is acted on: after a precondition or goal message, the next attempt gets past the reported failing action in 14.5% (ACT) and 15.8% (STATE) of turns, against 7.2% for BIN and 4.5% for SELF, and the mean index of the first failing action rises from 1.9 to 2.5 (ACT) and 2.6 (STATE) by round 3 while it stays at 1.9 under BIN. APPL goes furthest: with the applicable actions listed, the next attempt clears the reported action in 53.6% of turns, verbatim repetition falls to 4%, the mean failing index reaches 3.4, and four last attempts execute to the end without reaching the goal, yet only three instances are repaired. The revised plans then fail one or two actions later with the same class of error, so the error taxonomy at the last attempt is precondition failure in 105 to 120 of 126 cases for every arm. SELF is the only arm that introduces new error types: in 9 last attempts a state fact written in prose is parsed as an action, and 153 of its 378 outputs hit the 420-token cap because the model appends a step-by-step “verification” that usually finds no error.

Cost. Richer feedback is not expensive here. Generated tokens per repair turn are 64 (BIN), 68 (ACT), 62 (STATE) and 67 (APPL) against 192 for SELF; prompt length in round 3 is 1,033 tokens for BIN, 1,153 for ACT, 1,288 for STATE, 1,375 for APPL and 1,445 for SELF. On these small domains the full state adds about 45 tokens per round and the applicable actions another 30, still below the verbose self-verification arm.

Limitations. This is a small study under a fixed compute budget: one 4B model, greedy decoding, one prompt wording per arm, three repair rounds, and unequal instance counts across domains, so per-domain numbers are exploratory. The feedback levels differ in length as well as content; without a length-matched uninformative control we cannot separate the effect of a longer, structured message from that of its information. The validator is our own STRIPS executor, self-tested but not VAL. IPC and PlanBench instances are public, so Blocksworld memorization cannot be excluded; the fresh Mystery variant is unaffected.

References

Yusra Alkhazraji, Matthias Frorath, Markus Grützner, Malte Helmert, Thomas Liebetraut, Robert Mattmüller, Manuela Ortlieb, Jendrik Seipp, Tobias Springenberg, Philip Stahl, and Jan Wülfing. Pyperplan. Zenodo, 2020. <https://doi.org/10.5281/zenodo.3700819>.

- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.
- Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Llms can’t plan, but can help planning in llm-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, PMLR 235, 2024.
- Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. Llm+p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL – the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, 1998.
- Drew M. McDermott. The 1998 AI planning systems competition. *AI Magazine*, 21(2):35–55, 2000. doi: 10.1609/aimag.v21i2.1506.
- Bhavya Sai Nukapotula, Samin Moosavi, Haoze Wang, Luke Duncan, Diya Shakkottai, Varun Murali, and Srinivas Shakkottai. EvoPlan: Evolutionary neuro-symbolic robot planning with spatio-temporal guarantees. *arXiv preprint arXiv:2607.06724*, 2026.
- Shreyas Sundara Raman, Vanya Cohen, Ifrah Idrees, Eric Rosen, Ray Mooney, Stefanie Tellex, and David Paulius. Cape: Corrective actions from precondition errors using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. *arXiv:2211.09935*.
- Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B. Tenenbaum, Leslie Pack Kaelbling, and Michael Katz. Generalized planning in pddl domains with pretrained large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024.
- Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. *arXiv preprint arXiv:2402.08115*, 2024.
- Karthik Valmeekam, Matthew Marquez, and Subbarao Kambhampati. Can large language models really improve by self-critiquing their own plans? *arXiv preprint arXiv:2310.08118*, 2023a.
- Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, *Datasets and Benchmarks Track*, 2023b.
- Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models – a critical investigation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023c.
- Karthik Valmeekam, Kaya Stechly, and Subbarao Kambhampati. Llms still can’t plan; can llms? a preliminary evaluation of openai’s o1 on planbench. *arXiv preprint arXiv:2409.13373*, 2024.
- Pulkit Verma, Ngoc La, Anthony Favier, Swaroop Mishra, and Julie A. Shah. Teaching llms to plan: Logical chain-of-thought instruction tuning for symbolic planning. *arXiv preprint arXiv:2509.13351*, 2025.

Kaustubh Vyas, Damien Graux, Sébastien Montella, Pavlos Vougiouklis, Ruofei Lai, Keshuang Li, Yang Ren, and Jeff Z. Pan. An extensive evaluation of PDDL capabilities in off-the-shelf LLMs. *arXiv preprint arXiv:2502.20175*, 2025.

An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Jiajing Zhang, Jiamei Jiang, Chenyang Zhang, Feifei Mo, Linjing Li, and Daniel Zeng. Towards reliable and robust llm planning: Symbolic feedback-driven iterative self-refinement framework. *arXiv preprint arXiv:2606.27757*, 2026.

Zhehua Zhou, Jiayang Song, Kunpeng Yao, Zhan Shu, and Lei Ma. Isr-llm: Iterative self-refined large language model for long-horizon sequential task planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2081–2088, 2024.

A Prompts and feedback templates

Every conversation is rendered with the model’s chat template (generation prompt appended, thinking disabled). It consists of the system message, the round-0 user message, the round-0 assistant output stored verbatim (including any text outside the plan), and then, for each repair round, one user message carrying the feedback and the assistant reply. The round-0 user message is assembled from the task instruction and the PDDL files with comments and blank lines removed. In the listing below, the text in angle brackets is replaced by file contents; everything else is literal. Paragraphs are separated by one blank line; the two instruction paragraphs are single lines in the code and are wrapped here only for display.

```
[system]
You are a classical planning assistant.
```

```
[user, round 0]
Solve the following planning problem written in PDDL. Output a valid plan
for the problem: a sequence of actions that is applicable from the initial
state and reaches the goal.
```

```
Output format: one action per line, each written as a parenthesized PDDL
action, for example (action-name arg1 arg2). Output only the plan, with no
numbering and no commentary.
```

```
Domain (PDDL):
<domain file>
```

```
Example problem (PDDL):
<demonstration problem file>
```

```
Example plan:
<demonstration plan, one action per line>
```

```
Now solve this problem (PDDL):
<problem file>
```

Every repair message is a single line that ends with the same closing request, written as T below. SELF and BIN messages do not depend on the validator output. ACT messages depend on the error type; STATE messages equal the ACT message plus one sentence listing the state, which exists only for precondition and goal failures. In the listing, k is the 1-based index of the failing action among the parsed actions, A is that action as parsed (lower-cased, for example (stack b a)), n is the number of parsed actions, and every list of literals is the lexicographically sorted list of ground atoms separated by single spaces. Line breaks inside a message are for display only.

```
T: Output a complete corrected plan that starts from the initial state,
    one action per line, with no commentary.
```

```
SELF (all rounds, all error types):
Verify your plan step by step: for each action, check that its
```

preconditions hold in the current state and apply its effects; finally check that every goal condition holds. If you find an error, fix it. T

BIN (all rounds, all error types):

Your plan is invalid. T

ACT and STATE, unparseable:

Your response did not contain a plan in the required format. T

ACT and STATE, unknown-action:

Your plan is invalid. Action k, A, uses an action name that does not exist in this domain. T

ACT and STATE, wrong-arity:

Your plan is invalid. Action k, A, has the wrong number of arguments for this action. T

ACT and STATE, unknown-object (also used for invalid groundings):

Your plan is invalid. Action k, A, refers to an object that does not exist in this problem or is not a valid grounding. T

ACT, precondition:

Your plan is invalid. Action k, A, is not applicable in the state where it is executed: the precondition(s) <unmet preconditions> do not hold. T

STATE, precondition:

Your plan is invalid. Action k, A, is not applicable in the state where it is executed: the precondition(s) <unmet preconditions> do not hold. The true facts just before action k are: <all true facts>. T

ACT, goal-unmet:

Your plan is invalid. All n actions are applicable, but the goal is not achieved at the end: the goal condition(s) <unmet goal literals> are false in the final state. T

STATE, goal-unmet:

Your plan is invalid. All n actions are applicable, but the goal is not achieved at the end: the goal condition(s) <unmet goal literals> are false in the final state. The true facts at the end of the plan are: <all true facts>. T

APPL, precondition (goal-unmet analogously; other errors as in STATE):

Your plan is invalid. Action k, A, is not applicable in the state where it is executed: the precondition(s) <unmet preconditions> do not hold. The true facts just before action k are: <all true facts>. The actions applicable in that state are: <grounded applicable actions, alphabetical, at most 40, then "and m more">. T

As a concrete example, the Blocksworld demonstration problem has the initial state (clear b) (clear d) (handempty) (on a c) (on d a) (ontable b) (ontable c) and the goal (on a c) (on b d) (on d a). For the plan (pick-up b), (stack b a) the validator produces the following messages (closing request omitted).

ACT: Your plan is invalid. Action 2, (stack b a), is not applicable in the state where it is executed: the precondition(s) (clear a) do not hold.

STATE: Your plan is invalid. Action 2, (stack b a), is not applicable in the state where it is executed: the precondition(s) (clear a) do not hold. The true facts just before action 2 are: (clear d) (holding b) (on a c) (on d a) (ontable c).

For the plan (pick-up b), (put-down b) the messages are

ACT: Your plan is invalid. All 2 actions are applicable, but the goal

is not achieved at the end: the goal condition(s) (on b d) are false in the final state.

STATE: Your plan is invalid. All 2 actions are applicable, but the goal is not achieved at the end: the goal condition(s) (on b d) are false in the final state. The true facts at the end of the plan are: (clear b) (clear d) (handempty) (on a c) (on d a) (ontable b) (ontable c).

In Mystery Blocksworld the executor runs on the obfuscated task, so all action names and literals in ACT and STATE messages use the obfuscated vocabulary. No message ever contains a reference plan, a suggested action, or the number of remaining steps.

B Instances and reference plans

Sources. IPC instances are taken from the blocks, gripper, and logistics00 directories of the downward-benchmarks collection¹ [McDermott, 2000]. PlanBench basic Blocksworld instances 1 to 40 (domain blocksworld-4ops, four blocks each) are taken from the PlanBench repository [Valmeekam et al., 2023b]. Mystery instances are obfuscated copies of the selected Blocksworld instances generated by us (see below), not the published PlanBench Mystery files.

Reference plans. A reference plan is computed for every problem file in the four directories with pyperplan [Alkhazraji et al., 2020]. A* with the LM-cut heuristic is run in a subprocess that is killed after 40 s; if it returns, the plan is stored and marked optimal. Otherwise greedy best-first search with the FF heuristic is run with a 60 s limit and the plan is stored as non-optimal. Four subprocesses run in parallel on the CPU. Plan length is the number of actions. Mystery instances inherit the reference length and optimality flag of their Blocksworld original, since the two tasks are identical up to renaming; the Mystery demonstration plan, and the Mystery reference plans used in the self-test below, are the Blocksworld plans with the same renaming applied.

Selection rule. An instance is kept when its reference plan has at most 45 actions. Within each family the kept instance with the shortest reference plan, ties broken by the string order of the instance identifier, becomes the one-shot demonstration for all instances of that family and is removed from the evaluation set. In Blocksworld, PlanBench instances 5, 21, 31, and 34 all have two-action plans and instance-21 is selected by the tie-break. Mystery instances use the obfuscated copy of the Blocksworld demonstration. The evaluation set therefore contains 132 instances (Table 3).

Family	Source	<i>n</i>	Instances	Ref. length	Optimal
Blocksworld	IPC blocks	17	probBLOCKS-4-0 to probBLOCKS-9-2: three instances per size from 4 to 8 blocks, and 9-1, 9-2	6 to 28	17
Blocksworld	PlanBench	39	instance-1 to instance-40 except instance-21 (4 blocks each)	2 to 12	39
Mystery	obfuscated copies	56	the 56 Blocksworld instances above	2 to 28	56
Gripper	IPC gripper	4	prob02 to prob05 (6, 8, 10, 12 balls)	17 to 45	2
Logistics	IPC logistics00	16	16 instances listed in the text (15 to 29 objects)	14 to 45	10
All		132		2 to 45	124

Table 3: Evaluation instances. Ref. length is the range of reference-plan lengths; Optimal is the number of instances whose reference plan was found by A* with LM-cut. The demonstration instances (instance-21, its obfuscated copy, prob01, probLOGISTICS-5-2) are not included.

The IPC Blocksworld instance probBLOCKS-9-0 (reference plan of 64 actions) and all instances with 10 or more blocks (48 to 194 actions) are excluded by the length rule. The 16 Logistics instances are probLOGISTICS-4-0, 4-1, 4-2, 5-0, 5-1, 6-0, 6-1, 6-2, 6-9, 7-0, 7-1, 8-0, 9-0, 9-1, 10-1, and 12-0, with 15, 22, or 29 objects. The eight non-optimal references, for which A* timed out and the plan comes from greedy best-first search, are Gripper prob04 (37 actions) and prob05 (45), and Logistics 7-0 (37), 7-1 (45), 8-0 (34), 9-0 (38), 10-1 (43), and 12-0 (45). Plan-length ratios on these instances are relative to a satisficing plan and are marked as such wherever they are reported.

¹<https://github.com/aibasael/downward-benchmarks>

Demonstration plans. The four demonstrations are the reference plans of the demonstration instances.

Blocksworld, pb_blocks/instance-21:	Mystery, mystery_pb/instance-21:
(pick-up b)	(harangue b)
(stack b d)	(quorble b d)
Gripper, gripper/prob01:	Logistics, logistics00/probLOGISTICS-5-2:
(pick ball4 rooma left)	(load-truck obj23 tru2 pos2)
(pick ball1 rooma right)	(load-truck obj12 tru1 pos1)
(move rooma roomb)	(load-truck obj21 tru2 pos2)
(drop ball1 roomb right)	(drive-truck tru1 pos1 apt1 cit1)
(drop ball4 roomb left)	(unload-truck obj12 tru1 apt1)
(move roomb rooma)	(drive-truck tru2 pos2 apt2 cit2)
(pick ball2 rooma right)	(unload-truck obj23 tru2 apt2)
(pick ball3 rooma left)	(unload-truck obj21 tru2 apt2)
(move rooma roomb)	
(drop ball2 roomb right)	
(drop ball3 roomb left)	

Obfuscation. Mystery Blocksworld files are produced from the Blocksworld domain, problem, and demonstration files by removing comments and replacing every whole token in Table 4, case-insensitively (the IPC domain is named BLOCKS in its files); a token is delimited by characters other than letters, digits, underscores, and hyphens, so blocks inside a problem name such as BLOCKS-4-0 is left unchanged. Object names, problem names, and the file structure are unchanged. The same mapping is applied to the demonstration plan. The mapping is our own and differs from the published PlanBench Mystery vocabulary.

Actions		Predicates	
pick-up	harangue	on	nlooma
put-down	tinker	ontable	jaskur
stack	quorble	clear	vrytt
unstack	drimble	handempty	brakk
		holding	zunk
Domain names			
blocksworld-4ops	mystery-4ops	blocks	mystery

Table 4: Renaming used to build Mystery Blocksworld from the Blocksworld files.

C Validator

Grounded task. Domain and problem are parsed and grounded with pyperplan, which lower-cases all names. Its grounding (i) treats a predicate that occurs in no action effect as static, checks static preconditions once at grounding time, and drops static facts from operator preconditions and from the state; (ii) creates no operator for an argument tuple whose static preconditions are false in the initial state, for example a Logistics load-truck whose second argument is not a truck; and (iii) drops from every operator the add effects that are also among its preconditions and the delete effects that are also add effects, and then removes, by a relevance analysis backward from the goal, operators and effects that cannot contribute to any goal literal; a grounding left with no effect, such as moving from a room to itself, is removed in this step. Two consequences matter for reading the results. The facts listed by STATE are the facts of the grounded task, so type-like static facts such as (truck tru1) or (ball ball1) never appear. And a syntactically legal action that only changes goal-irrelevant facts, for example loading a Logistics package that has no goal condition, is reported as an invalid grounding rather than executed; on such wasted actions the executor is stricter than VAL. Reference plans contain no such actions.

Parsing. The raw model output is split into lines. Every line that contains a parenthesized group without nested parentheses contributes exactly one action: the first such group on the line, lower-cased and split on whitespace. Lines without such a group are ignored, so numbering, code fences, and

surrounding prose are tolerated, but any parenthesized text on a line is read as an action. Case is irrelevant because both the parsed tokens and the grounded task are lower-cased. If no line yields an action, the output is unparseable. Outputs cut off at the generation limit are validated as they are.

Execution and error taxonomy. Execution starts from the initial state of the grounded task, a set of facts, and processes the parsed actions in order. For action k the string (name arg1 ...) is looked up among the grounded operators. If no grounded operator matches this string, the categories are tested in this order: *unknown-action* if the action name is not an action of the domain; *wrong-arity* if the number of arguments differs from the action’s parameter count; *unknown-object* if some argument is neither an object of the problem nor a constant of the domain; otherwise *invalid-grounding*, which receives the same feedback message as unknown-object; the two labels are logged separately and merged when the taxonomy is reported. If the operator exists but its preconditions are not a subset of the current state, the category is *precondition*; the message reports the missing preconditions and, for STATE, the current state. Otherwise the state becomes (state minus delete effects) union add effects and execution continues. After the last action, if any goal literal is false the category is *goal-unmet*; the message reports the false goal literals and, for STATE, the final state. Otherwise the plan is *valid*. If no action was parsed the category is *unparseable*. The first failure ends the check, so the categories are mutually exclusive and every attempt receives exactly one label. Reported indices count parsed actions, not output lines.

Self-test. Every reference plan used in the study (the 132 evaluation instances and the 4 demonstrations, the Mystery copies obtained by renaming; 136 plans) is accepted by the executor, and every plan obtained from one of them by deleting a single action (1,707 plans) or by substituting a single argument of a single action by another object of the problem (40,498 plans) is rejected.

D Compute and reproducibility

Decoding. The model is loaded with Hugging Face transformers in bfloat16 with the SDPA attention kernel. Generation is greedy: sampling is disabled and temperature, top- p , and top- k are unset. Conversations are rendered with the tokenizer’s chat template with the generation prompt appended and thinking disabled; the tokenizer pads on the left and uses the end-of-sequence token as padding token if none is defined. At each generation step, the conversations awaiting a reply are sorted by the character length of the rendered text and processed in batches of 12 with at most 420 new tokens; a reply that reaches 420 tokens is flagged as truncated. Because decoding is greedy, the round-0 reply is produced once per instance and inserted verbatim as the first assistant turn of every arm.

Logging. One record is written per (instance, arm, round) with the number of prompt tokens (non-padding tokens of the rendered conversation, chat-template tokens included), the number of generated tokens (non-padding), the truncation flag, the feedback text, the raw output, and the validation result (validity, error category, failing index and action, missing literals, number of parsed actions). Records are appended to a JSONL file from which the driver resumes. Round 0 requires 132 generations; each repair round requires at most $|E|$ generations per arm, since solved instances leave the loop.

Hardware and cost. All generation runs on a single 24 GB consumer GPU. Total GPU time for the main model, all arms included, is 52 minutes (43 minutes for the first four arms, 9 minutes for APPL). Reference plans are computed on the CPU with four parallel pyperplan processes.

Analysis. Bootstrap intervals use 10,000 instance-level resamples with a fixed seed (NumPy default generator, seed 0); the exact binomial test on discordant pairs uses SciPy. Cumulative success at round r counts an instance as solved if its round-0 plan is valid or any repair round up to r under that arm produced a valid plan.

Reproduction. The supplementary archive contains the code (instance selection, reference plans, planning loop, analysis), all PDDL files including the obfuscated copies, the instance list with reference lengths and demonstration fields, the reference plans, the raw records, and the summaries. The pipeline is: compute reference plans; build the instance list with a 45-action limit (`-maxlen 45`); run the planning loop with three repair rounds and 420 new tokens (`-rounds 3 -max_new`

420); run the analysis, which evaluates three rounds by default. The limit, round count and token cap are command-line arguments; the script defaults (36 actions, 4 rounds, 360 tokens) were not used. Requirements are PyTorch, transformers, pyperplan, NumPy, SciPy, and Matplotlib.